SQL-Based Customer and Sales Data Analysis.

Presented by

Yogitha R

INTRODUCTION

This project focuses on analyzing customer, product, and order data using SQL.

The database contains separate information about customers, products, and their purchase transactions.

SQL is used to connect these tables and extract meaningful insights related to products, sales, customer behavior, and orders. purchase, product performance

OBJECTIVES

- 01 — Customer Analysis Understand customer demographics and purchasing behaviour.
- 02 — Product Analysis Analyze products based on category and price.
- 03 — Order Analysis Examine order quantity, payment methods and transaction patterns.
- 04 — Sales Analysis Calculate category-wise sales quantity and revenue.
- 05 — Relational Analysis Connect customer, product and order information using JOINs.
- 06 — Advanced SQL Analysis Apply subqueries and window functions to identify rankings and patterns.



CATEGORY-WISE TOTAL PRODUCTS ANALYSIS

```
SELECT category,
COUNT(product_id) AS
total_products
FROM PRODUCTS
GROUP BY category
HAVING
COUNT(product_id) >
10;
```



category	total_products
Home	21
Electronics	13
Clothing	15
Sports	21

DISPLAY PRODUCTS USING LIMIT AND OFFSET

```
SELECT
product_id, product_name,category,price
FROM PRODUCTS
ORDER BY price DESC
LIMIT 2 OFFSET 3;
```



product_id	product_name	category	price
P042	Smart Watch 42	Clothing	285.64
P047	USB Cable 47	Sports	277.58
NULL	NULL	NULL	NULL

CUSTOMER AND ORDER DETAILS USING INNER JOIN

```
SELECT
C.customer_id,C.gender,C.ci
ty,O.order_id,O.product_id,
O.quantity
FROM CUSTOMERS C
INNER JOIN ORDERS O
ON C.customer_id =
O.customer_id;
```

customer_id	city	order_id	product_id	quantity
C0692	London	O00001	P006	4
C1072	Nottingham	O00002	P059	5
C1187	Birmingham	O00003	P030	1
C0290	London	O00004	P063	1
C0094	Bristol	O00005	P032	2
C0103	Nottingham	O00006	P040	1
C0350	Birmingham	O00007	P076	1
C1019	Birmingham	O00008	P077	5
C0416	Nottingham	O00009	P055	3
C0824	Manchester	O00010	P042	1
C1118	Sheffield	O00011	P018	3

PRODUCT AND ORDER DETAILS USING LEFT JOIN

```
SELECT
P.product_id,P.product_name,P
.category,O.order_id,O.quantity
,O.payment_method
FROM PRODUCTS P
LEFT JOIN ORDERS O
ON                    P.product_id=
O.product_id;
```

product_id	product_name	category	order_id	quantity	payment_method
P001	Blender 1	Beauty	O00017	2	Card
P001	Blender 1	Beauty	O00049	4	Card
P001	Blender 1	Beauty	O00058	1	Cash
P001	Blender 1	Beauty	O00276	3	Online
P001	Blender 1	Beauty	O00370	2	Cash
P001	Blender 1	Beauty	O00451	1	Cash
P001	Blender 1	Beauty	O00460	5	Cash
P001	Blender 1	Beauty	O00470	2	Online
P001	Blender 1	Beauty	O00478	4	Online
P001	Blender 1	Beauty	O00674	2	Cash
P001	Blender 1	Beauty	O00686	4	Online

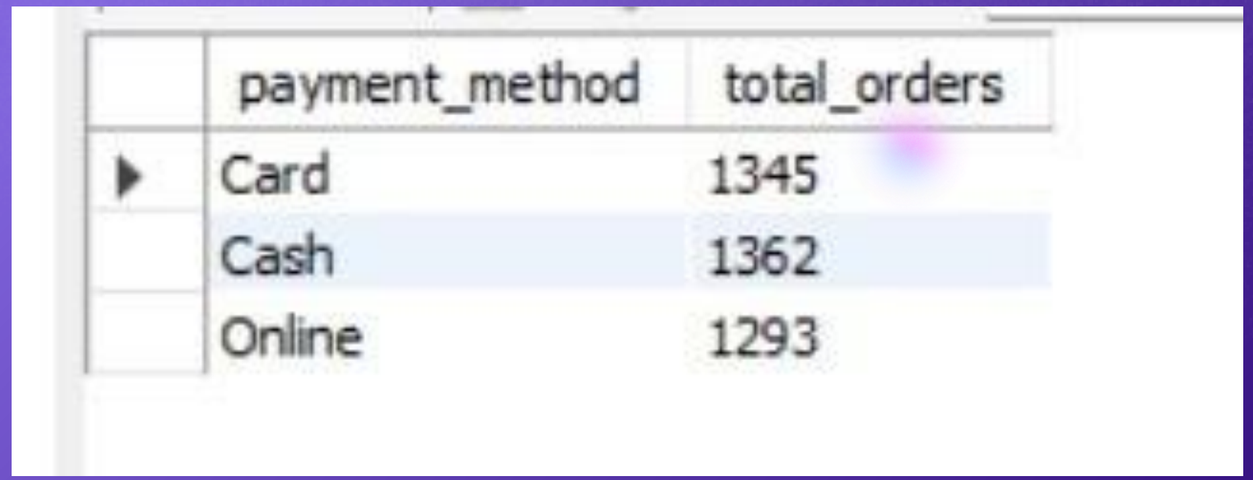
CUSTOMER AND ORDER DETAILS USING RIGHT JOIN

```
SELECT C.customer_id,C.city,O.order_id,
O.product_id,O.quantity
FROM CUSTOMERS C
RIGHT JOIN ORDERS O
ON C.customer_id = O.customer_id;
```

customer_id	city	order_id	product_id	quantity
C0692	London	O00001	P006	4
C1072	Nottingham	O00002	P059	5
C1187	Birmingham	O00003	P030	1
C0290	London	O00004	P063	1
C0094	Bristol	O00005	P032	2
C0103	Nottingham	O00006	P040	1
C0350	Birmingham	O00007	P076	1
C1019	Birmingham	O00008	P077	5
C0416	Nottingham	O00009	P055	3
C0824	Manchester	O00010	P042	1
C1118	Sheffield	O00011	P018	3

PAYMENT METHOD-WISE ORDER ANALYSIS

```
SELECT payment_method,COUNT(order_id) AS
total_orders
FROM ORDERS
GROUP BY payment_method;
```



payment_method	total_orders
Card	1345
Cash	1362
Online	1293

Category-wise Total Quantity

```
SELECT P.category,SUM(O.quantity) AS total_quantity
FROM PRODUCTS P
INNER JOIN ORDERS O
ON P.product_id = O.product_id
GROUP BY P.category
ORDER BY total_quantity DESC;
```

category	total_quantity
Sports	3250
Home	3074
Clothing	2101
Electronics	1851
Beauty	1623

Category-wise Revenue Analysis

```
SELECT P.category,SUM(O.quantity * P.price) AS
total_revenue
FROM PRODUCTS P
INNER JOIN ORDERS O
ON P.product_id = O.product_id
GROUP BY P.category
ORDER BY total_revenue DESC;
```

category	total_revenue
Home	473002.14
Sports	403963.89
Clothing	319942.70
Beauty	239438.96
Electronics	208470.02

Cumulative Price Calculation Using Window Function

```
SELECT
product_id,product_name,category,price,SUM(price)
OVER(PARTITION BY category ORDER BY price ASC) AS
cumulative_price
FROM PRODUCTS;
```

product_id	product_name	category	price	cumulative_price
P063	Jacket 63	Beauty	11.07	11.07
P041	Wireless Mouse 41	Beauty	36.66	47.73
P038	Keyboard 38	Beauty	40.13	87.86
P006	Vacuum Cleaner 6	Beauty	59.03	146.89
P065	USB Cable 65	Beauty	75.50	222.39
P078	USB Cable 78	Beauty	196.59	418.98
P034	Running Shoes 34	Beauty	233.44	652.42
P001	Blender 1	Beauty	272.76	925.18
P030	Coffee Maker 30	Beauty	276.23	1201.41
P079	Vacuum Cleaner 79	Beauty	289.84	1491.25
P075	Blender 75	Clothing	50.58	50.58

Previous Customer Age Using LAG Function

```
SELECT customer_id,gender,age,city,LAG(age)
OVER(ORDER BY customer_id) AS previous_age
FROM CUSTOMERS;
```

customer_id	gender	age	city	previous_age
C0001	Male	19	Liverpool	NULL
C0002	Male	65	Manchester	19
C0003	Female	20	London	65
C0004	Male	50	London	20
C0005	Female	32	Nottingham	50
C0006	Male	62	Sheffield	32
C0007	Male	31	Bristol	62
C0008	Female	24	Bristol	31
C0009	Female	20	Nottingham	24
C0010	Male	53	Liverpool	20
C0011	Male	63	Manchester	53

Next Customer Age Using LEAD Function

```
SELECT customer_id,gender,age,city,LEAD(age)
OVER(ORDER BY customer_id) AS next_age
FROM CUSTOMERS;
```

customer_id	gender	age	city	next_age
C0001	Male	19	Liverpool	65
C0002	Male	65	Manchester	20
C0003	Female	20	London	50
C0004	Male	50	London	32
C0005	Female	32	Nottingham	62
C0006	Male	62	Sheffield	31
C0007	Male	31	Bristol	24
C0008	Female	24	Bristol	20
C0009	Female	20	Nottingham	53
C0010	Male	53	Liverpool	63
C0011	Male	63	Manchester	36

Product Price Grouping Using NTILE

```
SELECT
product_id,product_name,category,price,NTILE(3)
OVER(ORDER BY price DESC) AS price_group
FROM PRODUCTS;
```

product_id	product_name	category	price	price_group
P079	Vacuum Cleaner 79	Beauty	289.84	1
P029	Face Cream 29	Home	287.29	1
P052	Laptop Stand 52	Home	287.13	1
P042	Smart Watch 42	Clothing	285.64	1
P047	USB Cable 47	Sports	277.58	1
P030	Coffee Maker 30	Beauty	276.23	1
P001	Blender 1	Beauty	272.76	1
P073	Water Bottle 73	Sports	269.97	1
P045	Perfume 45	Clothing	269.84	1
P011	Yoga Mat 11	Home	258.78	1
P070	T Shirt 70	Electronics	253.94	1

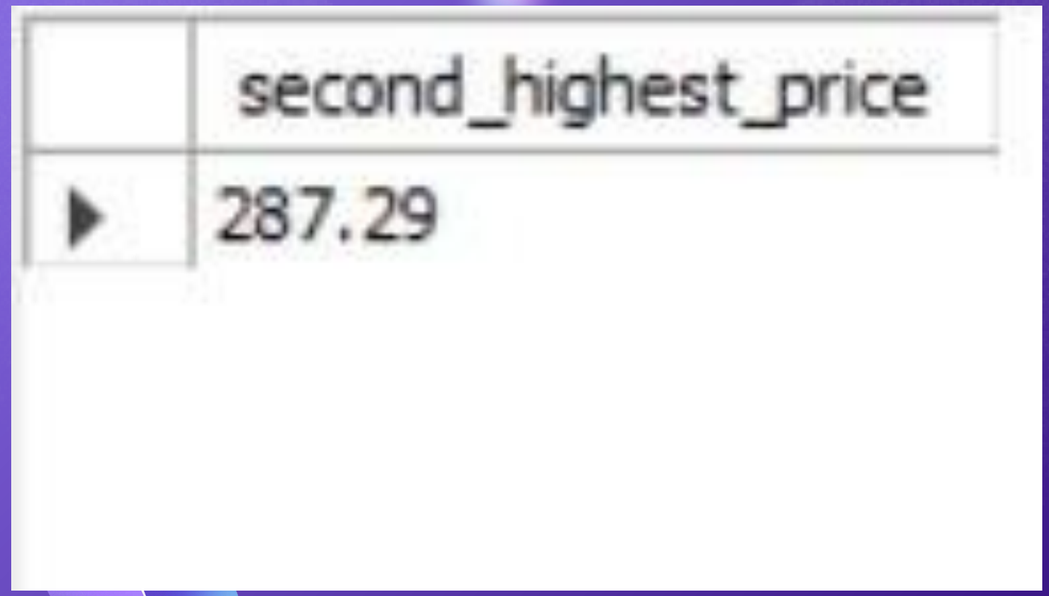
Row Number Generation Using ROW_NUMBER

```
SELECT
product_id,product_name,category,price,ROW_NUMBER()
OVER(ORDER BY product_id) AS row_index
FROM PRODUCTS;
```

product_id	product_name	category	price	row_index
P001	Blender 1	Beauty	272.76	1
P002	Perfume 2	Home	132.09	2
P003	Water Bottle 3	Home	58.58	3
P004	Smart Watch 4	Home	220.82	4
P005	Perfume 5	Home	27.81	5
P006	Vacuum Cleaner 6	Beauty	59.03	6
P007	Laptop Stand 7	Home	244.58	7
P008	Smart Watch 8	Electronics	93.18	8
P009	USB Cable 9	Clothing	156.62	9
P010	Yoga Mat 10	Home	48.44	10
P011	Yoga Mat 11	Home	258.78	11

Second Highest Product Price Using Subquery

```
SELECT MAX(price) AS second_highest_price
FROM PRODUCTS
WHERE price < (
   SELECT MAX(price)
   FROM PRODUCTS
);
```



	second_highest_price
▶	287.29

Age-wise Customer Ranking Analysis

```
SELECT customer_id,gender,age,city,RANK() OVER(ORDER
BY age DESC) AS age_rank
FROM CUSTOMERS;
```

customer_id	gender	age	city	age_rank
C0002	Male	65	Manchester	1
C0143	Male	65	Nottingham	1
C0235	Male	65	Sheffield	1
C0256	Male	65	London	1
C0302	Female	65	Nottingham	1
C0395	Female	65	London	1
C0428	Female	65	Bristol	1
C0437	Male	65	Leeds	1
C0469	Male	65	Birmingham	1
C0533	Male	65	Leeds	1
C0709	Male	65	Nottingham	1

CONCLUSION

- This SQL project helped analyze product, customer, and order data effectively.
- Different SQL concepts like JOINS, GROUP BY, HAVING, SUBQUERIES, and WINDOW FUNCTIONS were used for data analysis.
- The project improved understanding of database management and query writing.
- SQL analysis helps businesses make better decisions using data insights.
- This project enhanced practical knowledge in data analytics and reporting.

THANK YOU!